

LAPORAN DATA CLEANING & DATA PREPARATION

Shipping Logistics Database — Migrasi CSV ke Microsoft SQL Server (T-SQL)

Database: LogistikDB | Tabel Sumber: dbo.ShippingLogistics (23 kolom)

Sumber Data Awal: File CSV | Tools: Microsoft SQL Server Management Studio (SSMS), T-SQL, Visual Studio Code

Disusun oleh: Eko Damar Yogi

1. LATAR BELAKANG & TUJUAN PROJECT

Data pada project ini berawal dari sebuah file CSV mentah berisi catatan transaksi pengiriman barang (shipment), yang kemudian di-import dan dikonversi menjadi tabel database pada Microsoft SQL Server dengan nama database LogistikDB dan tabel sumber dbo.ShippingLogistics. Tabel ini memiliki granularitas satu baris per pengiriman (ShipmentSK), mencakup 23 kolom yang merekam detail order, rute pengiriman (kota asal/tujuan, jarak), spesifikasi muatan (berat, volume), pihak yang terlibat (carrier, warehouse, driver), biaya, estimasi vs realisasi durasi pengiriman, status pengiriman, alasan keterlambatan, hingga rating dari pelanggan.

Karena berasal dari proses migrasi CSV ke database relasional, data mentah pada tabel ini masih memuat sejumlah masalah kualitas data khas hasil import file datar (flat file), mulai dari missing value pada berbagai kolom, baris duplikat, hingga inkonsistensi format teks pada kolom kota yang biasanya luput dari validasi saat proses input CSV. Laporan ini mendokumentasikan proses Data Cleaning & Preparation yang dilakukan sepenuhnya menggunakan T-SQL di lingkungan SQL Server, dengan tujuan menghasilkan tabel akhir dbo.ShippingLogistics_Cleaned yang bersih, konsisten, dan siap digunakan untuk kebutuhan analisis maupun pelaporan (reporting-ready).

Prinsip Kerja yang Diterapkan

- **Non-destructive:** seluruh proses cleaning dilakukan pada tabel salinan (ShippingLogistics_Cleaning), bukan langsung pada tabel sumber, sehingga data asli tetap utuh sebagai cadangan.
- **Root-cause driven:** setiap missing value ditelusuri korelasinya dengan kolom lain (misalnya DeliveryStatus) sebelum ditentukan metode imputasinya, bukan langsung dihapus atau diisi angka default.
- **Terdokumentasi:** setiap keputusan cleaning (imputasi, penghapusan duplikat, standardisasi teks) dicatat alasannya agar dapat diaudit ulang di kemudian hari.

2. RINGKASAN DATASET SUMBER

Tabel dbo.ShippingLogistics hasil konversi dari file CSV memiliki 23 kolom dengan granularitas satu baris per pengiriman (ShipmentSK sebagai surrogate key, dan OrderID sebagai identifier order). Struktur lengkap kolomnya adalah sebagai berikut:

Kolom	Tipe Informasi	Keterangan
ShipmentSK	Identifier	Surrogate key unik untuk setiap baris pengiriman.
OrderID	Identifier	ID order pengiriman (dapat memiliki lebih dari satu shipment terkait).
OrderDate	Tanggal	Tanggal order dibuat.
PickupDate	Tanggal	Tanggal barang diambil (pickup) dari pengirim.
DeliveryDate	Tanggal	Tanggal barang diantarkan ke tujuan.
OriginCity	Teks	Kota asal pengiriman.
DestinationCity	Teks	Kota tujuan pengiriman.
DistanceKM	Numerik	Jarak tempuh pengiriman (kilometer).
WeightKG	Numerik	Berat muatan barang (kilogram).
VolumeM3	Numerik	Volume muatan barang (meter kubik).
Carrier	Kategorikal	Perusahaan/mitra jasa pengiriman (carrier).
ShippingType	Kategorikal	Jenis layanan pengiriman (misalnya reguler/ekspres).
VehicleType	Kategorikal	Jenis kendaraan yang digunakan untuk pengiriman.
CustomerType	Kategorikal	Jenis pelanggan (misalnya individu/bisnis).

Kolom	Tipe Informasi	Keterangan
PaymentMethod	Kategorikal	Metode pembayaran biaya pengiriman.
WarehouseID	Identifier	ID gudang asal pengiriman diproses.
DriverID	Identifier	ID pengemudi yang bertugas mengantarkan.
ShippingCostIDR	Numerik	Total biaya pengiriman (dalam Rupiah).
FuelSurchargeIDR	Numerik	Biaya tambahan bahan bakar (dalam Rupiah).
PlannedDeliveryDays	Numerik	Estimasi jumlah hari pengiriman yang direncanakan.
ActualDeliveryDays	Numerik	Jumlah hari aktual proses pengiriman.
DeliveryStatus	Kategorikal	Status pengiriman, di antaranya Delivered, Delayed, Returned, dan Lost.
DelayReason	Kategorikal	Alasan keterlambatan pengiriman (jika ada).
CustomerRating	Numerik (opsional)	Rating dari pelanggan terhadap layanan pengiriman.

Tabel 1. Struktur Kolom pada `dbo.ShippingLogistics` (23 kolom, hasil migrasi dari CSV).

Dari 23 kolom tersebut, proses cleaning pada laporan ini berfokus pada kolom-kolom yang teridentifikasi bermasalah, yaitu `DelayReason`, `DeliveryDate`, `ActualDeliveryDays`, `VehicleType`, `PaymentMethod`, `FuelSurchargeIDR`, `CustomerRating` (missing value), `OrderID` (baris duplikat), serta `OriginCity` dan `DestinationCity` (inkonsistensi format teks) — sebagaimana dijelaskan pada bagian berikut.

3. METODOLOGI DATA CLEANING

Proses data cleaning dilakukan secara bertahap melalui sembilan langkah berikut, seluruhnya dijalankan menggunakan T-SQL di lingkungan Microsoft SQL Server:

1. Duplikasi tabel sumber ke tabel kerja (`ShippingLogistics_Cleaning`) untuk menjaga keutuhan data asli.
2. Pemeriksaan menyeluruh jumlah missing value pada setiap kolom menggunakan dynamic SQL.
3. Investigasi dan penanganan missing value pada kolom `DelayReason`.
4. Investigasi missing value pada kolom `DeliveryDate` dan `ActualDeliveryDays`.
5. Investigasi dan penanganan missing value gabungan pada `VehicleType`, `PaymentMethod`, dan `FuelSurchargeIDR`.
6. Keputusan mempertahankan NULL pada kolom `CustomerRating`.
7. Deteksi dan penghapusan baris duplikat berdasarkan `OrderID`.
8. Standardisasi format teks pada kolom `OriginCity` dan `DestinationCity`.
9. Pembuatan tabel akhir hasil cleaning (`ShippingLogistics_Cleaned`).

4. TAHAPAN DATA CLEANING SECARA DETAIL

4.1 Duplikasi Tabel Kerja (Backup untuk Proses Cleaning)

Sebelum melakukan perubahan apa pun, tabel sumber disalin terlebih dahulu ke tabel kerja agar data asli tidak berubah selama proses cleaning berlangsung.

```
DROP TABLE IF EXISTS dbo.ShippingLogistics_Cleaning;

SELECT *
```

```
INTO dbo.ShippingLogistics_Cleaning
FROM dbo.ShippingLogistics;
```

4.2 Pemeriksaan Missing Value pada Seluruh Kolom (Dynamic SQL)

Alih-alih memeriksa kolom satu per satu secara manual, digunakan pendekatan dynamic SQL dengan memanfaatkan INFORMATION_SCHEMA.COLUMNS untuk secara otomatis membangun query COUNT(*) WHERE kolom IS NULL bagi setiap kolom pada tabel, kemudian digabungkan dengan UNION ALL dan dieksekusi melalui sp_executesql. Pendekatan ini membuat proses audit missing value menjadi lebih efisien dan dapat digunakan kembali untuk tabel lain.

```
DECLARE @sql NVARCHAR(MAX);

SELECT @sql = STRING_AGG(
    'SELECT ''' + COLUMN_NAME + ''' AS column_name,
        COUNT(*) AS missing_count
    FROM dbo.ShippingLogistics_Cleaning
    WHERE ' + QUOTENAME(COLUMN_NAME) + ' IS NULL',
    ' UNION ALL '
)
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'dbo'
AND TABLE_NAME = 'ShippingLogistics_Cleaning';

EXEC sp_executesql @sql;
```

Hasil pemeriksaan ini mengarahkan tim data untuk menelusuri lebih lanjut empat kolom yang teridentifikasi memiliki missing value: DelayReason, DeliveryDate/ActualDeliveryDays, VehicleType/PaymentMethod/FuelSurchargeIDR, dan CustomerRating.

4.3 Penanganan Missing Value: DelayReason

Kolom DelayReason pada awalnya menunjukkan jumlah missing value yang sangat ekstrem, sehingga ditelusuri korelasinya dengan kolom DeliveryStatus:

```
SELECT DeliveryStatus, COUNT(*) AS missing_count
FROM dbo.ShippingLogistics_Cleaning
WHERE DelayReason IS NULL
GROUP BY DeliveryStatus;
```

Temuan (Root Cause)

Seluruh missing value pada DelayReason berkorelasi penuh dengan DeliveryStatus = 'Delivered'. Artinya, kekosongan ini bukan data yang hilang secara tidak sengaja, melainkan konsekuensi logis: pengiriman yang berhasil tanpa delay memang tidak memiliki alasan keterlambatan untuk dicatat.

Berdasarkan temuan tersebut, missing value ditangani dengan imputasi label yang mencerminkan kondisi sebenarnya:

```
UPDATE dbo.ShippingLogistics_Cleaning
SET DelayReason = 'No Delay'
WHERE DelayReason IS NULL AND DeliveryStatus = 'Delivered';
```

4.4 Investigasi Missing Value: DeliveryDate & ActualDeliveryDays

```
SELECT deliverydate, actualdeliverydays, deliverystatus
FROM dbo.ShippingLogistics_Cleaning
WHERE deliverydate IS NULL;
```

Temuan (Root Cause)

Missing value pada kedua kolom ini seluruhnya berasal dari order dengan DeliveryStatus = 'Lost' — barang yang hilang dalam proses pengiriman memang tidak pernah memiliki tanggal pengantaran maupun durasi pengiriman aktual.

Karena NULL di sini merepresentasikan kondisi nyata (barang tidak pernah sampai), kedua kolom sengaja dibiarkan NULL dan tidak diimputasi dengan nilai apa pun, untuk menghindari informasi yang menyesatkan pada analisis durasi pengiriman.

4.5 Penanganan Missing Value Gabungan: VehicleType, PaymentMethod, FuelSurchargeIDR

Ketiga kolom ini diperiksa bersamaan karena pola missing value-nya diduga saling terkait:

```
SELECT
  SUM(CASE WHEN VehicleType IS NULL THEN 1 ELSE 0 END) AS VehicleType_NULL,
  SUM(CASE WHEN PaymentMethod IS NULL THEN 1 ELSE 0 END) AS PaymentMethod_NULL,
  SUM(CASE WHEN FuelSurchargeIDR IS NULL THEN 1 ELSE 0 END) AS FuelSurcharge_NULL,
  SUM(CASE
    WHEN VehicleType IS NULL
      AND PaymentMethod IS NULL
      AND FuelSurchargeIDR IS NULL
    THEN 1 ELSE 0
  END) AS Combined_NULL
FROM dbo.ShippingLogistics_Cleaning;
```

Temuan (Root Cause)

Terkonfirmasi bahwa missing value pada VehicleType, PaymentMethod, dan FuelSurchargeIDR terjadi pada 120 baris yang sama persis — mengindikasikan ketiga informasi ini kemungkinan tidak tercatat pada satu titik input data yang sama (misalnya order yang diinput melalui kanal/sistem tertentu yang belum lengkap datanya).

a) VehicleType & PaymentMethod (kategorikal) → diberi label 'Unknown'

```
UPDATE dbo.ShippingLogistics_Cleaning
SET VehicleType = 'Unknown',
    PaymentMethod = 'Unknown'
WHERE VehicleType IS NULL OR PaymentMethod IS NULL;
```

b) FuelSurchargeIDR (numerik) → diperiksa lebih lanjut, lalu diisi median

```
SELECT DeliveryStatus, COUNT(*) AS missing_count
FROM dbo.ShippingLogistics_Cleaning
WHERE FuelSurchargeIDR IS NULL
GROUP BY DeliveryStatus;
```

Missing value pada FuelSurchargeIDR ditemukan tersebar pada order dengan status Delayed, Delivered, dan Returned, bukan pola yang mengindikasikan kondisi nyata seperti pada kasus DelayReason, sehingga dipilih pendekatan median imputation:

```
UPDATE dbo.ShippingLogistics_Cleaning
SET FuelSurchargeIDR = (
    SELECT DISTINCT PERCENTILE_CONT(0.5) WITHIN GROUP
        (ORDER BY FuelSurchargeIDR) OVER()
    FROM dbo.ShippingLogistics_Cleaning
    WHERE FuelSurchargeIDR IS NOT NULL
)
WHERE FuelSurchargeIDR IS NULL;
```

Median dipilih dibanding rata-rata (mean) karena lebih tahan terhadap outlier pada biaya bahan bakar yang cenderung bervariasi tergantung jarak dan jenis kendaraan.

4.6 CustomerRating: Dibiarkan NULL

Berbeda dengan kolom-kolom sebelumnya, CustomerRating merupakan data feedback yang sifatnya opsional/tidak wajib diisi oleh pelanggan. Oleh karena itu, missing value pada kolom ini tidak diimputasi dan tetap dibiarkan NULL, karena mengisi rating yang sebenarnya tidak diberikan pelanggan justru akan mendistorsi hasil analisis kepuasan pelanggan di kemudian hari.

4.7 Deteksi & Penghapusan Baris Duplikat

Duplikasi diperiksa berdasarkan OrderID, mengingat setiap OrderID seharusnya unik untuk satu transaksi pengiriman:

```
SELECT *
FROM dbo.ShippingLogistics_Cleaning
WHERE OrderID IN (
    SELECT OrderID
    FROM dbo.ShippingLogistics_Cleaning
    GROUP BY OrderID
    HAVING COUNT(*) > 1
)
ORDER BY OrderID;
```

Temuan (Root Cause)

Sebagian besar OrderID duplikat memiliki nilai yang identik di seluruh kolom sehingga dapat langsung dihapus. Namun, ditemukan dua pasang order duplikat dengan nilai ShippingCostIDR yang berbeda signifikan, mengindikasikan potensi kesalahan input data (data entry error) pada salah satu baris.

Untuk kasus dengan nilai ShippingCostIDR berbeda, baris dengan nilai yang lebih rendah dipertahankan sebagai nilai yang lebih wajar, menggunakan teknik ROW_NUMBER() untuk deduplikasi:

```
WITH DuplicateCheck AS (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY OrderID
            ORDER BY ShippingCostIDR ASC
        ) AS row_num
    FROM dbo.ShippingLogistics_Cleaning
)
DELETE FROM DuplicateCheck
WHERE row_num > 1;
```

Query verifikasi berikut dijalankan setelah proses penghapusan untuk memastikan tidak ada lagi OrderID yang duplikat:

```
SELECT OrderID, COUNT(*) AS duplicate_count
```

```
FROM dbo.ShippingLogistics_Cleaning
GROUP BY OrderID
HAVING COUNT(*) > 1;
```

4.8 Standardisasi Format Teks: OriginCity & DestinationCity

Kedua kolom kota ditemukan memiliki inkonsistensi format, campuran huruf besar/kecil, dan spasi berlebih, yang berpotensi menyebabkan kesalahan pengelompokan (grouping) pada analisis berbasis kota jika tidak distandardisasi.

```
UPDATE dbo.ShippingLogistics_Cleaning
SET OriginCity = TRIM(UPPER(OriginCity)),
    DestinationCity = TRIM(UPPER(DestinationCity));
```

4.9 Pembuatan Tabel Akhir Hasil Cleaning

Setelah seluruh tahapan di atas selesai, tabel kerja disalin menjadi tabel final `dbo.ShippingLogistics_Cleaned` yang siap digunakan untuk analisis lanjutan:

```
DROP TABLE IF EXISTS dbo.ShippingLogistics_Cleaned;

SELECT * INTO dbo.ShippingLogistics_Cleaned
FROM dbo.ShippingLogistics_Cleaning;
```

5. RINGKASAN MASALAH DATA & METODE PENANGANAN

Tabel berikut merangkum seluruh isu kualitas data yang ditemukan beserta root cause dan metode penanganannya:

Kolom Terdampak	Jenis Masalah	Root Cause	Metode Penanganan
DelayReason	Missing value (ekstrem)	Order dengan status Delivered memang tidak memiliki alasan keterlambatan	Imputasi label 'No Delay'
DeliveryDate, ActualDeliveryDays	Missing value	Barang berstatus Lost tidak pernah memiliki tanggal/durasi pengantaran	Dibiarkan NULL (mencerminkan kondisi nyata)
VehicleType, PaymentMethod	Missing value (120 baris, kombinasi)	Data tidak tercatat pada titik input yang sama	Imputasi label 'Unknown'
FuelSurchargeIDR	Missing value (120 baris, kombinasi)	Tersebar di berbagai status pengiriman, bukan pola sistematis	Median imputation
CustomerRating	Missing value	Field opsional dari feedback pelanggan	Dibiarkan NULL (tidak diimputasi)
OrderID (baris)	Baris duplikat	Kemungkinan double-entry pada proses input; sebagian dengan ShippingCostIDR berbeda	Deduplikasi via ROW_NUMBER(), simpan ShippingCostIDR terendah
OriginCity, DestinationCity	Inkonsistensi format teks	Input manual tanpa standardisasi (spasi, huruf besar/kecil campur)	TRIM + UPPER pada seluruh isi kolom

Tabel 2. Ringkasan Isu Data, Root Cause, dan Metode Penanganan.

6. KESIMPULAN & REKOMENDASI

Kesimpulan

Proses data cleaning terhadap tabel `dbo.ShippingLogistics` berhasil mengidentifikasi dan menangani **tiga kategori masalah utama**: (1) missing value pada lima kolom dengan pola dan root cause yang berbeda-beda, (2) baris duplikat berdasarkan OrderID — termasuk kasus dengan nilai biaya pengiriman yang tidak konsisten, dan (3) inkonsistensi format teks pada kolom kota. Setiap keputusan penanganan didasarkan pada investigasi korelasi antar kolom terlebih dahulu (**root-cause driven**), bukan langsung menghapus atau mengisi nilai secara default, sehingga integritas makna data tetap terjaga. Hasil akhir proses ini adalah tabel `dbo.ShippingLogistics_Cleaned` yang lebih konsisten dan siap digunakan untuk analisis performa logistik (on-time delivery rate, biaya pengiriman per kota, dsb.).

Rekomendasi

- **Validasi input di sisi sumber (hulu)**: terapkan validasi wajib isi (mandatory field) pada VehicleType, PaymentMethod, dan FuelSurchargeIDR di sistem input agar kombinasi missing value pada 120 baris tersebut tidak berulang di periode berikutnya.
- **Constraint UNIQUE pada OrderID**: menambahkan primary key/unique constraint pada kolom OrderID di tabel sumber untuk mencegah duplikasi data masuk sejak awal.
- **Standardisasi di level aplikasi**: terapkan dropdown/lookup list untuk OriginCity dan DestinationCity (alih-alih input teks bebas) guna mencegah inkonsistensi format di masa depan.
- **Audit berkala**: jalankan kembali query pemeriksaan missing value (Tahap 4.2) secara rutin/terjadwal untuk memantau kualitas data secara berkelanjutan, bukan hanya sekali di awal project.
- **Dokumentasi metadata**: catat definisi resmi setiap DeliveryStatus (termasuk kapan DelayReason boleh kosong) sebagai referensi bagi tim data di masa depan agar penanganan missing value konsisten.

7. LAMPIRAN: FULL SCRIPT T-SQL

Berikut adalah script T-SQL lengkap yang digunakan sepanjang proses data cleaning ini, dapat dijalankan ulang secara berurutan di SQL Server Management Studio (SSMS):

```
USE LogistikDB;

SELECT TOP 10 *
FROM dbo.ShippingLogistics;

SELECT COUNT(*) AS total_OrderID_Unique
FROM dbo.ShippingLogistics;

SELECT COUNT(DISTINCT OrderID) AS total_rows
FROM dbo.ShippingLogistics;

-- duplicate table for cleaning and transformation
DROP TABLE IF EXISTS dbo.ShippingLogistics_Cleaning;

SELECT *
INTO dbo.ShippingLogistics_Cleaning
FROM dbo.ShippingLogistics;

SELECT TOP 10 *
FROM dbo.ShippingLogistics_Cleaning;
```

```

SELECT COUNT(*) AS total_rows
FROM dbo.ShippingLogistics_Cleaning;

DECLARE @sql NVARCHAR(MAX);

SELECT @sql = STRING_AGG(
    'SELECT ''' + COLUMN_NAME + ''' AS column_name,
        COUNT(*) AS missing_count
    FROM dbo.ShippingLogistics_Cleaning
    WHERE ' + QUOTENAME(COLUMN_NAME) + ' IS NULL',
    ' UNION ALL '
)
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'dbo'
AND TABLE_NAME = 'ShippingLogistics_Cleaning';

EXEC sp_executesql @sql;

SELECT TOP 10 DelayReason
FROM dbo.ShippingLogistics_Cleaning
GROUP BY DelayReason;

SELECT TOP 10 DeliveryStatus, DelayReason
FROM dbo.ShippingLogistics_Cleaning;

SELECT DeliveryStatus, COUNT(*) AS missing_count
FROM dbo.ShippingLogistics_Cleaning
WHERE DelayReason IS NULL
GROUP BY DeliveryStatus;

UPDATE dbo.ShippingLogistics_Cleaning
SET DelayReason = 'No Delay'
WHERE DelayReason IS NULL AND DeliveryStatus = 'Delivered';

SELECT COUNT(*) AS delayreason_count_missing
FROM dbo.ShippingLogistics_Cleaning
WHERE DelayReason IS NULL;

SELECT deliverydate, actualdeliverydays, deliverystatus
FROM dbo.ShippingLogistics_Cleaning
WHERE deliverydate IS NULL;

SELECT TOP 10 *
FROM dbo.ShippingLogistics_Cleaning
WHERE FuelSurchargeIDR IS NULL;

SELECT
    SUM(CASE WHEN VehicleType IS NULL THEN 1 ELSE 0 END) AS VehicleType_NULL,
    SUM(CASE WHEN PaymentMethod IS NULL THEN 1 ELSE 0 END) AS PaymentMethod_NULL,
    SUM(CASE WHEN FuelSurchargeIDR IS NULL THEN 1 ELSE 0 END) AS FuelSurcharge_NULL,
    SUM(CASE
        WHEN VehicleType IS NULL
        AND PaymentMethod IS NULL
        AND FuelSurchargeIDR IS NULL
        THEN 1 ELSE 0
    END) AS Combined_NULL
FROM dbo.ShippingLogistics_Cleaning;

UPDATE dbo.ShippingLogistics_Cleaning
SET VehicleType = 'Unknown',
    PaymentMethod = 'Unknown'
WHERE VehicleType IS NULL OR PaymentMethod IS NULL;

```

```
SELECT DeliveryStatus, COUNT(*) AS missing_count
FROM dbo.ShippingLogistics_Cleaning
WHERE FuelSurchargeIDR IS NULL
GROUP BY DeliveryStatus;

UPDATE dbo.ShippingLogistics_Cleaning
SET FuelSurchargeIDR = (
    SELECT DISTINCT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY FuelSurchargeIDR) OVER()
    FROM dbo.ShippingLogistics_Cleaning
    WHERE FuelSurchargeIDR IS NOT NULL
)
WHERE FuelSurchargeIDR IS NULL;

SELECT *
FROM dbo.ShippingLogistics_Cleaning
WHERE OrderID IN (
    SELECT OrderID
    FROM dbo.ShippingLogistics_Cleaning
    GROUP BY OrderID
    HAVING COUNT(*) > 1
)
ORDER BY OrderID;

WITH DuplicateCheck AS (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY OrderID
            ORDER BY ShippingCostIDR ASC
        ) AS row_num
    FROM dbo.ShippingLogistics_Cleaning
)
DELETE FROM DuplicateCheck
WHERE row_num > 1;

SELECT OrderID, COUNT(*) AS duplicate_count
FROM dbo.ShippingLogistics_Cleaning
GROUP BY OrderID
HAVING COUNT(*) > 1;

UPDATE dbo.ShippingLogistics_Cleaning
SET OriginCity = TRIM(UPPER(OriginCity)),
    DestinationCity = TRIM(UPPER(DestinationCity));

SELECT TOP 10 * FROM dbo.ShippingLogistics_Cleaning;

DROP TABLE IF EXISTS dbo.ShippingLogistics_Cleaned;
SELECT * INTO dbo.ShippingLogistics_Cleaned
FROM dbo.ShippingLogistics_Cleaning;
```